

VR Mini Project-2 Report

T.V.S. Chaitanya
IMT2022545

Tahir Khadrabad
IMT2022100

Anirudh Pathaneni
IMT2022505

May 18, 2025

1 Dataset Extraction

1.1 Data Processing and Metadata Integration

To prepare the dataset for Visual Question Answering (VQA), we first performed structured pre-processing by integrating image metadata with rich product information available in the Amazon Berkeley Objects (ABO) dataset. The goal was to create a unified table linking each image to a concise set of meaningful attributes such as product name, color, type, bullet points, and keywords.

1.2 Step-by-Step Processing Workflow

1.2.1 Loading Image Metadata

- We read the image metadata from `images.csv`, which includes image IDs and relative paths for images stored under `images/small/`.

1.2.2 Aggregating Product Metadata

- We parsed all JSON files of the format `listings_*.json` under the `listings/metadata/` directory.
- These files contain multilingual fields and attributes such as `item_name`, `color`, `product_type`, `bullet_point`, and `item_keywords`.

1.2.3 Feature Extraction and Normalization

- Extracted single values from nested fields using helper functions.
 - For example, `color` was extracted from the `standardized_values` list if available.
 - `item_name` was formatted into a multilingual string (e.g., “en: red shirt; fr: chemise rouge”).
- Converted keyword and bullet point lists to JSON-encoded strings for serialization.

1.2.4 Merging Image and Product Metadata

- Merged metadata on:
 - `main_image_id` (primary image)
 - `other_image_id` (secondary images, exploded to flatten the list)
- Labeled images based on whether they were primary or secondary (`image_source`) and ensured no duplication.

1.2.5 Data Cleaning and Validation

- Verified actual image existence on disk using the `Path.exists()` method.
- Retained only entries with valid and present images.

- Final dataset columns included:
 - `item_id`, `image_id`, `item_name`, `color`, `product_type`, `bullet_point`, `item_keywords`, `full_image_path`

1.2.6 Output

- The final processed metadata was saved to `merged_metadata.csv`.
- This CSV file serves as the foundation for automated question generation and subsequent VQA model training.

2 Data Curation Using Gemini API

To create a high-quality Visual Question Answering (VQA) dataset, we leveraged the Gemini 2.0 API to generate multimodal single-word answer questions. The curated questions were designed to closely align with the image content, constrained to only what could be inferred visually, with minimal reliance on auxiliary metadata.

2.1 Motivation and Design Principles

- We aimed to simulate human-like VQA tasks, where questions are grounded in what is observable in the image.
- Metadata was used only to guide question types and enhance relevance, but not to introduce any information that isn't visually inferable.

2.2 Tools and Infrastructure

- Gemini 2.0 API (Flash Lite model) for multimodal question generation.
- Pandas, PIL, and JSON for image and metadata handling.

2.3 Curation Pipeline Steps

2.3.1 Metadata Preparation

We used the preprocessed metadata (`merged_metadata.csv`) obtained during the earlier data integration step. It includes key fields like:

- `item_name`, `color`, `product_type`
- `full_image_path`, `image_id`

2.3.2 Prompt Design for Gemini 2.0

Each image was passed to the Gemini model along with a structured prompt that requested:

- Exactly 3 VQA questions with single-word answers
- 4 options per question (1 correct + 3 distractors)

Why Multiple-Choice Options Were Included:

- At first, we included MCQs because they reduced ambiguity in answers and made both training and evaluation more reliable and consistent for humans and models alike. But later we found out that because of using MCQs the model was overfitting, so later we stopped giving options to the model while training.

Sample Prompt (abridged):

```
# Prepare prompt
prompt = (
    "Generate exactly 3 Visual Question Answering (VQA) questions with  

    single-word answers for a multimodal VQA dataset."  

    "Questions must be such that one can answer just by looking at the  

    image, only using metadata for guidance or confirmation."  

    "Each question must have 4 multiple-choice options (the correct answer  

    plus 3 plausible distractors)."  

    "Return the response in strictly valid JSON format with fields:  

    question, answer, options (list of 4 strings).\n\n"  

    "Metadata:\n"  

    "- Item Name: {item_name}\n"  

    "- Color: {color}\n"  

    "- Product Type: {product_type}\n"  

    # "- Key Bullet Points: {bullet_text}\n"  

    # "item_keywords={keywords}"
)
```

2.3.3 Custom Post-Processing Logic

- We parsed and validated each JSON response for structure and plausibility.
- For robustness, we fallback-generated questions like:
 - “What is the color of the [product_type]?”
 - “What type of product is this?”
- This ensured every image had exactly 3 questions for uniform training data.

2.3.4 Batch Processing and Rate Limiting

- We processed the dataset in batches of 10, respecting Gemini’s API rate limits (15 RPM).
- Each batch was saved as a separate CSV file to ensure modularity and easy debugging.

2.4 Rejected Alternatives and Design Tradeoffs

Yes/No Questions: Initially, we considered including yes/no questions. However, our baseline model (e.g., `blip2-flan-t5-xl`) consistently predicted correct answers for such binary questions even without explicit yes/no examples in training. This suggested:

- The model had been pre-trained on general yes/no patterns.
- Including them would not significantly improve learning.

Hence, we focused exclusively on fact-based, single-word MCQs that would better leverage fine-tuning.

We intentionally excluded `bullet_point` and `item_keywords` from the prompts, as these fields often contain non-visual or promotional text that cannot be deduced from the image alone (e.g., “waterproof”, “ideal for travel”).

3 Project Structure and Repository

The project files are organized and hosted in a GitHub repository at <https://github.com/anirudhpathaneni/VR-MiniProject-2>, with the following structure based on recent commits:

- **IMT2022505_100_545/**: Directory containing inference scripts, added in the latest commit.
- **Blip-2-Base.ipynb**: Kaggle notebook with base model evaluation and metrics for **blip2-flan-t5-xl**.
- **Blip-vqa-base-Base.ipynb**: Kaggle notebook with base model evaluation and metrics for **blip-vqa-base**.
- **LoRA-Blip-vqa-base-Seq.ipynb**: Kaggle notebook with LoRA fine-tuning and metrics for **blip-vqa-base**.
- **README.md**: Initial project documentation.
- **blip-vqa-base_finetune_1.csv**: CSV file with fine-tuning metrics for **blip-vqa-base**.
- **blip-vqa-base_metrics_base.csv**: CSV file with base metrics for **blip-vqa-base**.
- **blip2-base_metrics_base.csv**: CSV file with base metrics for **blip2-flan-t5-xl**.
- **dc.py** and **dp.py**: Python scripts for data curation code and QA dataset generation.
- **vqa_dataset_final_new.csv**: Final curated QA dataset in CSV format.

The CSV files contain performance metrics for the models, while the Jupyter notebooks (‘.ipynb’) are Kaggle notebooks used for model training, evaluation, and experimentation.

4 Models Used

For this Visual Question Answering (VQA) project, we utilized two pre-trained models from Salesforce, each designed for vision-language tasks but with distinct architectures and strengths:

- **Salesforce/blip-vqa-base**: This model is a specialized vision-language model tailored for Visual Question Answering tasks. It combines a vision transformer (ViT) for image processing with a text encoder-decoder for question understanding and answer generation. Its training on VQA-specific datasets makes it adept at producing structured, grounded answers, particularly for questions requiring concise responses like those in our dataset.
- **Salesforce/blip2-flan-t5-xl**: This is a more general-purpose vision-language model that integrates a Vision Transformer with a Flan-T5-XL language model (an enhanced T5 variant). It uses a Q-Former module to align visual and textual features and is optimized for a broad range of tasks, including image captioning, instruction-following, and VQA. Its larger size (3B parameters) and general training make it versatile but potentially less specialized for concise VQA answers.

Both models were trained to fine-tune using Low-Rank Adaptation (LoRA) to adapt to our curated VQA dataset, with configurations optimized for Kaggle’s GPU constraints.

5 Evaluation Metrics and Model Performance Analysis

We evaluated each VQA model on our curated dataset using three key metrics:

- **Accuracy**: Measures the proportion of correct predictions.

- **F1 Score:** Harmonic mean of precision and recall — better reflects performance when class imbalance or ambiguity exists.
- **BERTScore:** Measures semantic similarity between predicted and ground truth answers using BERT embeddings, useful for evaluating short text or slightly varied correct answers.

Table 1: Model Performance Metrics

Model	Accuracy	F1 Score	BERTScore
blip-vqa-base	0.323236	0.088085	0.96276
blip2-flan-t5-xl	0.267772	0.09119	0.94183
LoRA-blip-vqa-base	0.67382	0.27207	0.98236
LoRA-blip2-flan-t5-xl	—	—	—

5.1 Model Comparison: blip-vqa-base vs. blip2-flan-t5-xl

- blip-vqa-base outperformed blip2-flan-t5-xl in accuracy and BERTScore.
- Why this happens:
 - blip-vqa-base is specifically trained for VQA tasks, including answer generation in structured, grounded ways.
 - blip2-flan-t5-xl is a more general vision-language model optimized for both image captioning and instruction-following tasks. It may not specialize in concise, single-word answers or classification-style VQA.
 - blip2-flan-t5-xl’s transformer-based design (with Q-Former and a frozen image encoder) may struggle with fine-grained visual attribute recognition like color or product type without additional tuning.

In summary, blip-vqa-base is more tailored to the format and type of questions in our dataset, while blip2-flan-t5-xl, being broader in capability, showed slightly poorer alignment with the dataset’s requirements.

6 Fine-Tuning blip-vqa-base Model with LoRA

To adapt the blip-vqa-base model to our curated Visual Question Answering dataset, we used parameter-efficient fine-tuning with Low-Rank Adaptation (LoRA) on Kaggle GPUs. We implemented quantization for memory efficiency, used XFormers for optimized attention, and designed decoding and training parameters for controlled and stable learning.

6.1 LoRA Configuration

We used the PEFT (Parameter-Efficient Fine-Tuning) library to apply LoRA, which injects trainable low-rank matrices into the attention mechanism. Our configuration was:

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["query", "value", "dense"],
    lora_dropout=0,
    bias="none",
```

)

Explanation:

- `r=8`: Rank of the low-rank matrices. Smaller ranks mean fewer trainable parameters.
- `lora_alpha=16`: Controls the scaling of the update. A higher value increases the learning capacity of LoRA layers.
- `target_modules=["query", "value", "dense"]`: We inserted LoRA adapters into the key transformer blocks responsible for attention and representation.
- `lora_dropout=0`: No dropout, since we relied on early stopping and regularization via weight decay.
- `bias="none"`: Biases were not adapted, only attention weights.

This configuration allowed us to train a much smaller number of parameters ($\sim 0.2\%$ of the original model), making it feasible within Kaggle's limited compute budget.

6.2 Quantization and Optimization Techniques

We loaded the model with 8-bit quantization using `BitsAndBytesConfig`, reducing memory usage and accelerating training:

```
quantization_config = BitsAndBytesConfig(
    load_in_8bit=True,
    bnb_8bit_compute_dtype=torch.bfloat16,
    bnb_8bit_use_double_quant=True,
    bnb_8bit_quant_type="nf8"
)
```

- `load_in_8bit=True`: Loads model weights in 8-bit.
- `bnb_8bit_use_double_quant`: Double quantization for further compression.
- `bnb_8bit_quant_type="nf8"`: A normalized format suitable for transformer models.

6.3 Attention Mechanism

We initially attempted Flash Attention, but it was not compatible with our hardware and `blip-vqa-base` implementation. Instead, we enabled XFormers Attention, which:

- Provides memory-efficient attention computation.
- Allows faster forward passes during both training and inference.

```
try:
    base_model.config.use_xformers_attention = True
except AttributeError:
    print("xformers attention not supported")
```

6.4 Training Arguments

```
training_args = Seq2SeqTrainingArguments(
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
```

```
num_train_epochs=3,
learning_rate=1e-4,
fp16=True,
bf16=False,
optim="adamw_8bit",
weight_decay=0.05,
report_to="wandb",
eval_strategy="epoch",
load_best_model_at_end=True,
predict_with_generate=True
)
```

Key Points:

- `fp16/bf16`: Mixed-precision training was enabled to reduce memory footprint and improve speed.
- `adamw_8bit`: Used 8-bit AdamW optimizer, a memory-efficient variant for large models.
- `learning_rate=1e-4`: Tuned for stability and convergence with LoRA updates.
- `load_best_model_at_end=True`: Ensures the most accurate model on validation set is retained.
- `predict_with_generate=True`: Required for generating answers rather than just classification logits.

We also logged progress and metrics using Weights & Biases (WandB).

6.5 Generation Parameters for Inference

```
generation_params = {
    "temperature": 0.3,
    "top_k": 40,
    "top_p": 0.8,
    "max_new_tokens": 5,
    "num_beams": 5,
    "no_repeat_ngram_size": 2,
    "use_cache": True,
    "do_sample": True
}
```

Purpose of each parameter:

- `temperature=0.3`: Low value ensures less randomness and more deterministic outputs, which is important for one-word answers.
- `top_k=40` and `top_p=0.8`: Sampling strategies to limit output to most probable tokens, improving fluency and diversity.
- `max_new_tokens=5`: Restricts output length, consistent with single-word answer requirement.
- `num_beams=5`: Beam search improves output quality by considering multiple possible sequences.

- `use_cache=True`: Enables KV Cache, reducing computation in decoder layers during generation.
- `do_sample=True`: Ensures sampling within top_k/top_p for variety in generated answers.

6.6 Observations

- The KV Cache was used successfully, improving inference efficiency.
- The XFormers attention gave better throughput than default PyTorch attention, though Flash Attention was incompatible.
- Quantization helped stay within Kaggle’s memory limits while enabling faster training.

6.7 Impact of Fine-Tuning with LoRA

- After applying Low-Rank Adaptation (LoRA) to the `blip-vqa-base` model, we observed significant improvements across all three metrics:
 - Accuracy increased from 32% $\rightarrow$ 67%
 - F1 Score tripled
 - BERTScore improved to 0.98, showing semantically more accurate predictions

Why Fine-Tuning with LoRA Improved Performance:

- *Task-specific adaptation*: LoRA fine-tuned the model on our curated VQA dataset, allowing it to adapt to the format, vocabulary, and structure of our questions and answers.
- *Low-rank efficient updates*: LoRA modifies only low-rank decomposed weight matrices, enabling efficient training even with limited compute, while still significantly improving model representation for our dataset.
- *Reduced overfitting risk*: Because LoRA updates fewer parameters, the model generalizes better, which likely explains the boost in F1 Score (which considers precision and recall, not just correct matches).
- *Multiple-choice alignment*: The format of our questions (MCQs with single-word answers) allowed the fine-tuned model to map visual features directly to a constrained set of valid outputs, improving classification-like behavior.

6.8 Result Summary and Challenges with `blip2-flan-t5-xl`

After fine-tuning, we evaluated the `blip-vqa-base` model using accuracy, macro F1-score, and BERTScore. The LoRA-tuned `blip-vqa-base` model achieved substantial improvements in all metrics, confirming the effectiveness of parameter-efficient fine-tuning for VQA tasks.

For `blip2-flan-t5-xl`, we encountered an `OutOfMemoryError: CUDA out of memory` during fine-tuning on Kaggle GPUs despite efforts to mitigate it. We reduced `per_device_train_batch_size` and `per_device_eval_batch_size` to lower values and set `lora_dropout=0.1`, but the issue persisted, limiting training to only 1 epoch. Further optimization strategies are required to fully fine-tune this larger model.